

Input

```
def mod_inverse(e, phi):  
    for d in range(1, phi):  
        if (e * d) % phi == 1:  
            return d
```

p = 61

q = 53

n = p * q

phi = (p - 1) * (q - 1)

e = 17

d = mod_inverse(e, phi)

print("Original Keys")

print("Public Key:", (e, n))

print("Private Key:", (d, n))

new_e = 31

new_d = mod_inverse(new_e, phi)

print("\nNew Keys Using Same n")

print("New Public Key:", (new_e, n))

print("New Private Key:", (new_d, n))

print("\nIf an attacker knows the old private key,")

print("they can recover $\phi(n)$ and compute future keys.")

print("Generate new p and q for security.")

Output

Original Keys

Public Key: (17, 3233)

Private Key: (2753, 3233)

New Keys Using Same n

New Public Key: (31, 3233)

New Private Key: (1711, 3233)

If an attacker knows the old private key,
they can recover $\phi(n)$ and compute future keys.
Generate new p and q for security.

|